Class 7 - **04 November 2026**

GenAI System Mindset — summary

Introduction to AI and Machine Learning

Executive Master in Artificial Intelligence for Business

Roberto Henriques · Fernando Peres

NOVA IMS INFORMATION MANAGEMENT SCHOOL

The building blocks of a GenAI system

CONTEXT (C)

"What the model needs to know, when it needs to know it, and how to provide it effectively."

TOOLS & INTEGRATIONS (T)

Extend model capabilities through external information and actions.

FLOWS

Coordinate multiple steps and actions to accomplish a goal.

- (W) WORKFLOW
- (A) AGENTS

GUARDRAILS (G)

Validate and constrain system behavior.

EVALS (E)

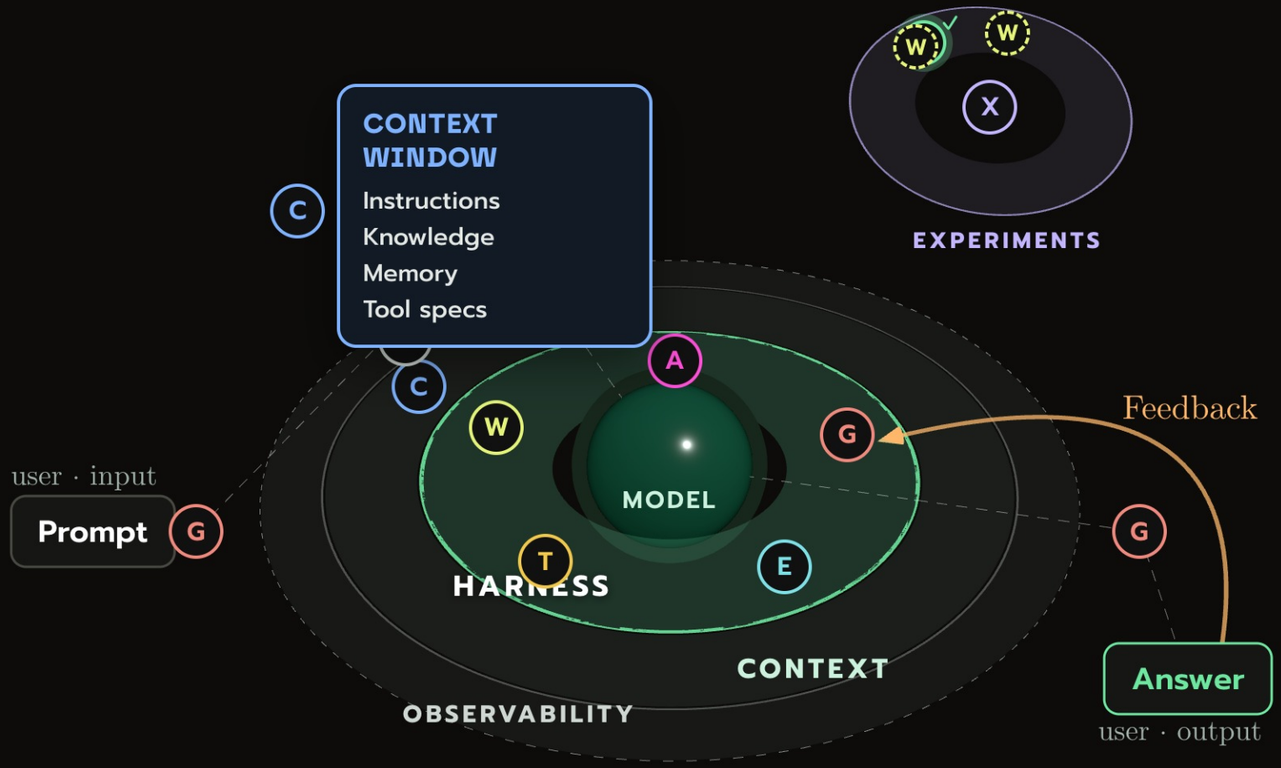
Assess quality, correctness, safety, and task success.

EXPERIMENTS (X)

Systematically test and compare models, prompts, tools, context strategies, workflows, and agents.

OBSERVABILITY (O)

Monitor and trace system execution, performance, and costs.



In the animation, O is a satellite: it orbits the observability ring, collecting traces, metrics and costs from every harness component.

Context: what the model needs to know

What the model needs to know, and how to provide it: business context, rules and knowledge. **When** it arrives is the harness's job.

ONE IDEA TO KEEP

A model can only be as good as what it sees. Context is where your data, rules and know-how enter the system.

Context engineering
the discipline: decide what the model should see

Context
the runtime asset the harness draws on

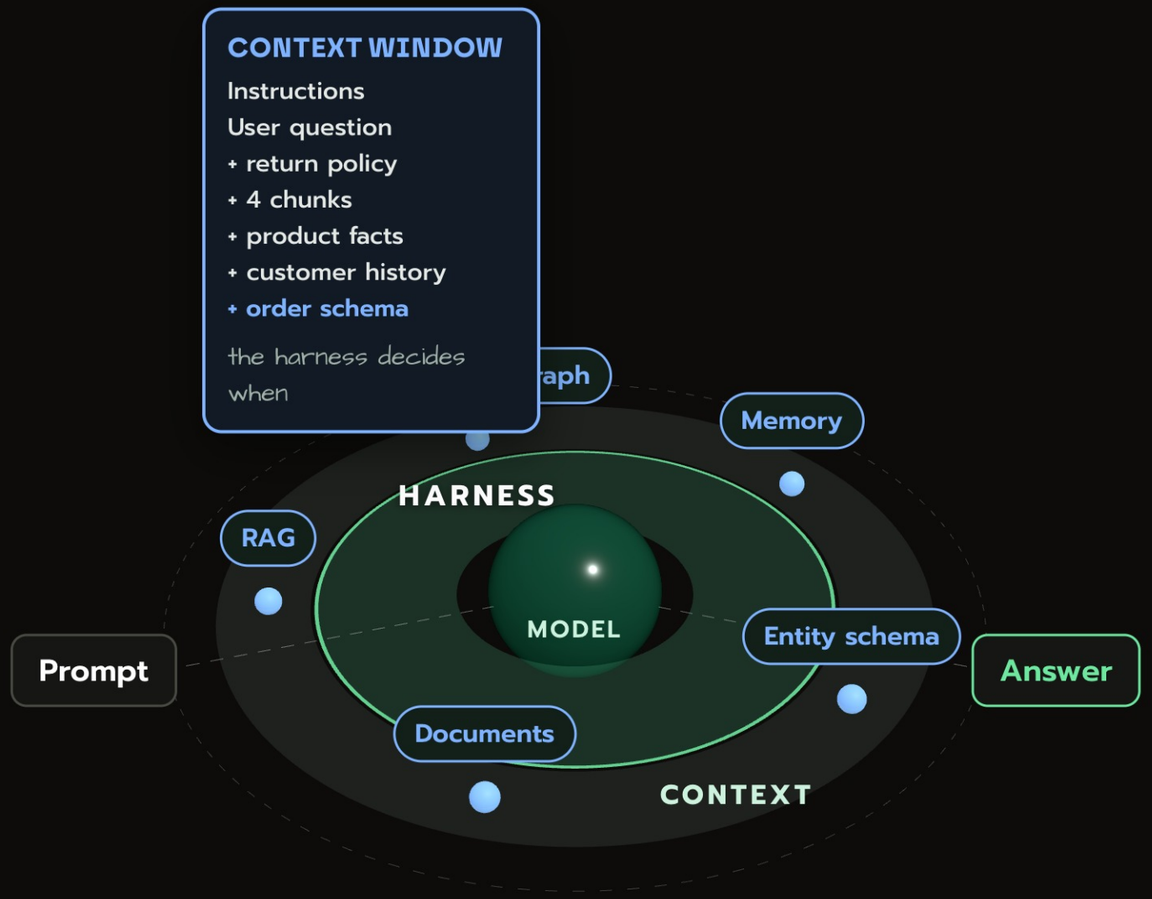
WHAT YOU HAVE

Curate it: documents, databases, policies, the team's know-how.

WHAT YOU DEFINE

Engineer it: entity, memory and input/output schemas, rules, prompts and skills, how to retrieve.

Documents · RAG · Knowledge graphs · Memory schema · Entity schemas · Ontologies



Tools: extend what the model can do

Extend model capabilities: process data, fetch information and take actions in other systems.

ONE IDEA TO KEEP

Connecting a model to tools does not make it smarter. It changes its reach, and with it, what can go wrong.

Tool

1 capability: one function, one job

API · MCP server

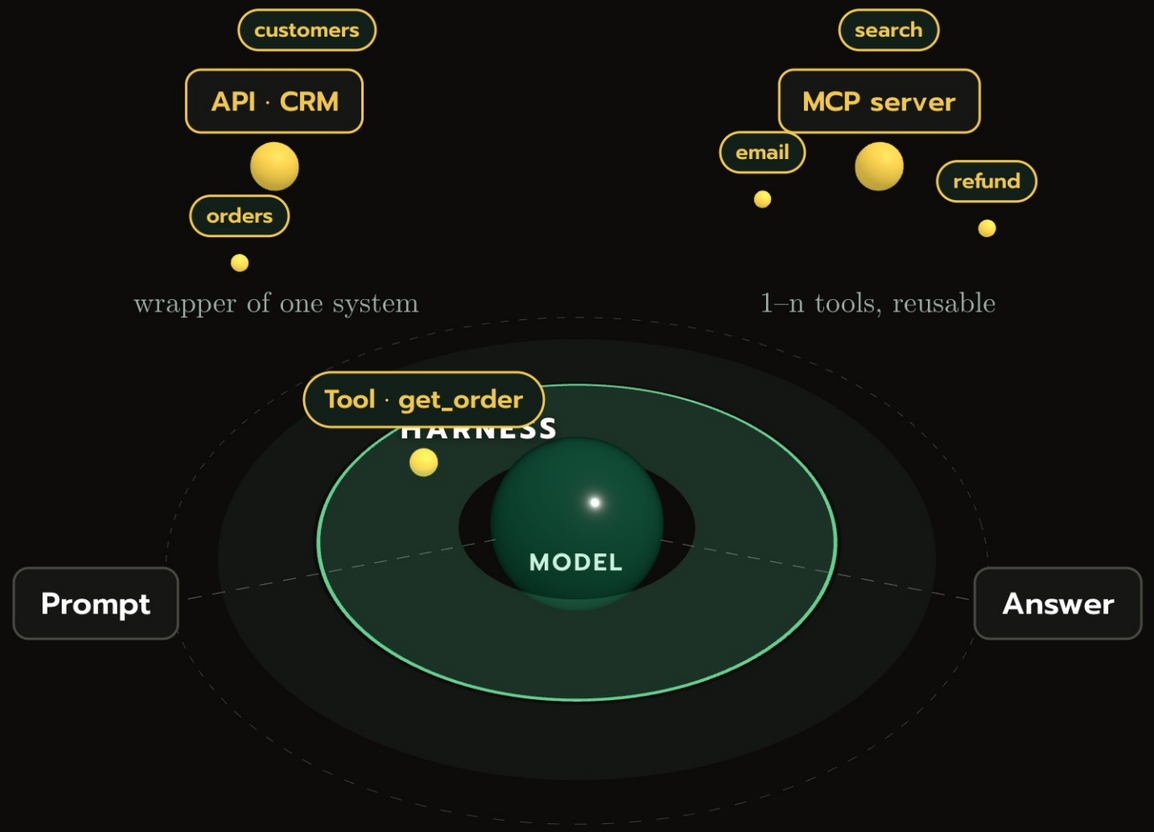
1–n capabilities: a reusable wrapper or proxy around tools

READ VS. ACT

Fetching data is low risk. Changing something, a refund or an email, needs limits.

BUILT-IN VS. CONNECTED

Your own functions, or a server many apps reuse: N + M integrations, not N × M.



TOOL RESULTS

get_order → #1042 shipped

Workflows: the same path, every time

Coordinate multiple steps and actions to accomplish a goal. In a workflow, **your code** fixes the order of the steps; the model does the language inside them.

ONE IDEA TO KEEP

A workflow follows the path you drew. Same input, same steps: easy to test, easy to explain.

Workflow · this slide
follows steps: code decides the path

Agent
autonomy: the model decides the path

WHEN TO USE

Known, repeatable tasks: classify, route, extract, draft then check.

FIVE COMMON PATTERNS

Chaining, routing, parallel, orchestrator-workers, evaluator-optimizer.

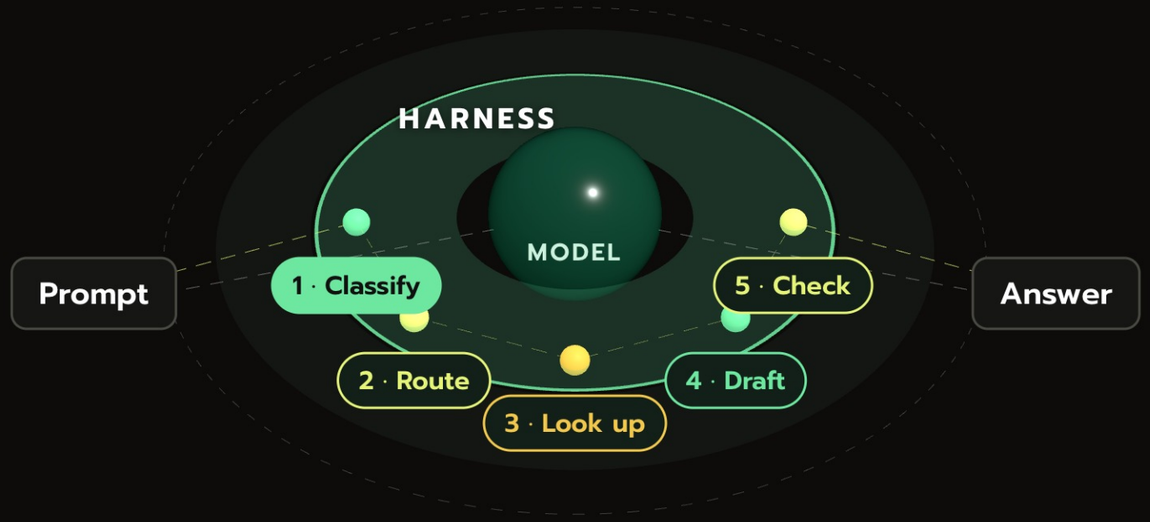
Patterns from Anthropic, Building effective agents (2024).

WORKFLOW

the same path, every time

Run 4: classify → route → look up → draft → check

● model ● code ● tool



Agents: the model chooses the path

Coordinate multiple steps and actions to accomplish a goal. An agent works in a loop, **think → act → observe**, and decides the next step itself.

AGENT

chooses its own path

Run 3: email → refund → orders

think: goal reached, stop

ONE IDEA TO KEEP

An agent chooses its own path. More reach for open-ended tasks, and more to check.

Workflow

follows steps: code decides the path

Agent · this slide

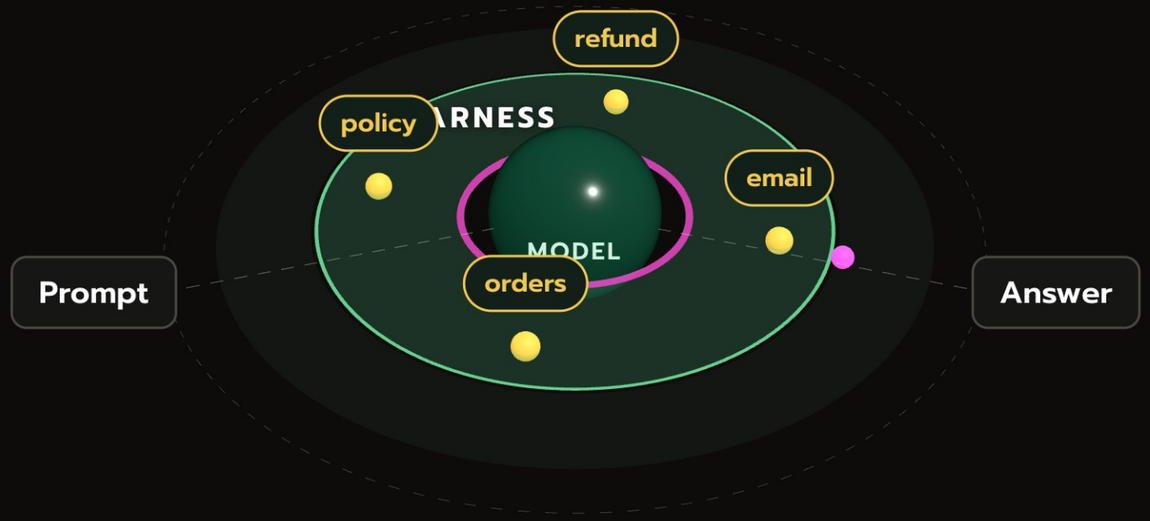
autonomy: the model decides the path

AGENT VS. AGENTIC

An **agent** is one loop that plans, uses tools and checks results. **Agentic** describes a whole system: agents and workflows with shared memory and controls.

SINGLE- VS. MULTI-AGENT

One loop with a few tools: start here. Several specialists, coordinated by a lead or by handoffs, when one is not enough.



Guardrails: checks in code around the model

Validate and constrain system behavior: rules that run on every request, before, during and after the model.

ONE IDEA TO KEEP

Asking the model to behave is not a control. Guardrails in code enforce the rule, whatever the model does.

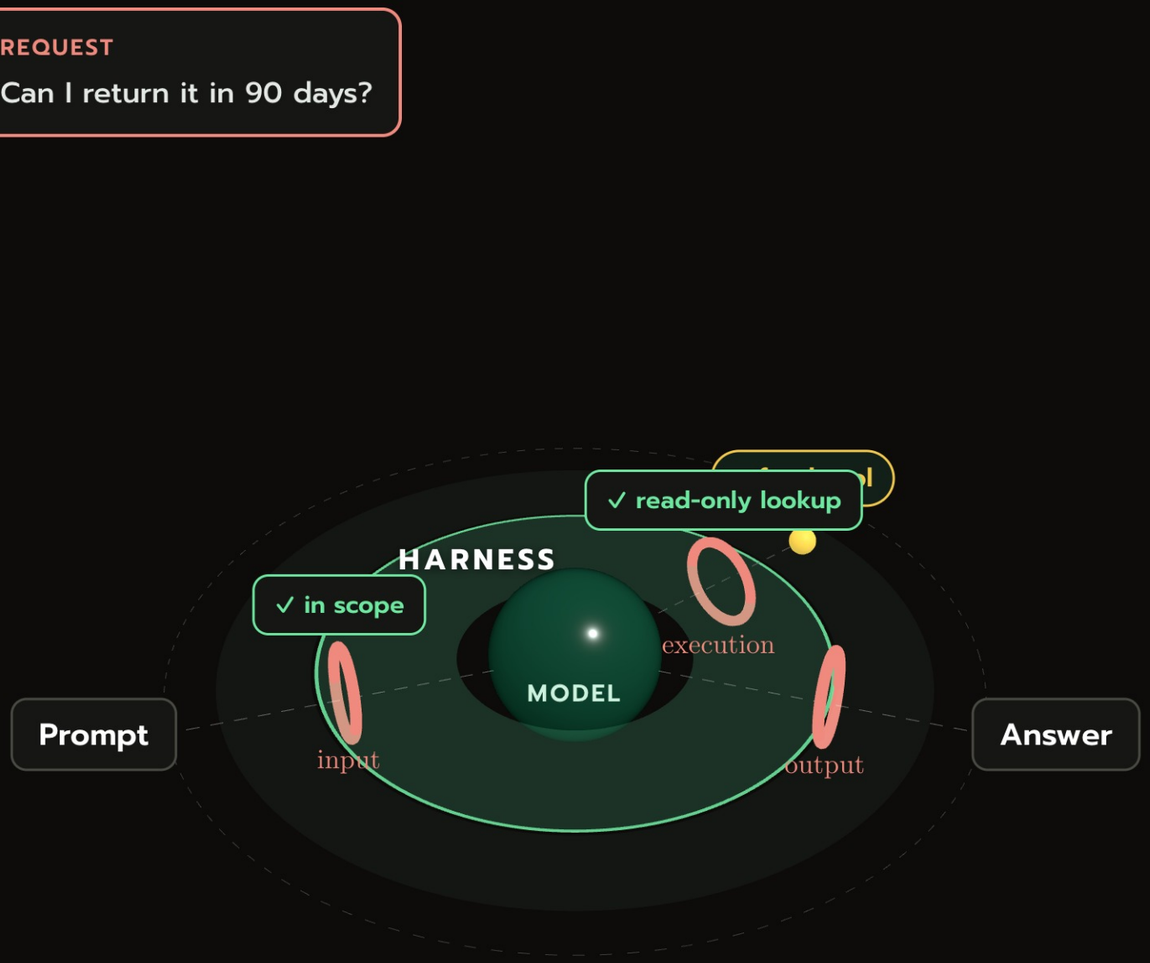
Input — what is allowed in

Execution — what it is allowed to do

Output — what is allowed out

- Personal-data redaction
- Prompt-injection detection
- Tool access by role
- Approval thresholds
- Groundedness
- Secret filter

Every guardrail has a cost: latency, false alarms and upkeep. Test them like the rest of the system.



Evals: evidence, not impressions

Assess quality, correctness, safety and task success: repeatable checks whose results you can compare over time.

ONE IDEA TO KEEP

No eval, no evidence. Evals decide whether a change ships, which model to use, and whether a guardrail works.

Code checks
fast and exact

LLM as a judge
reads like a person

Human review
the reference

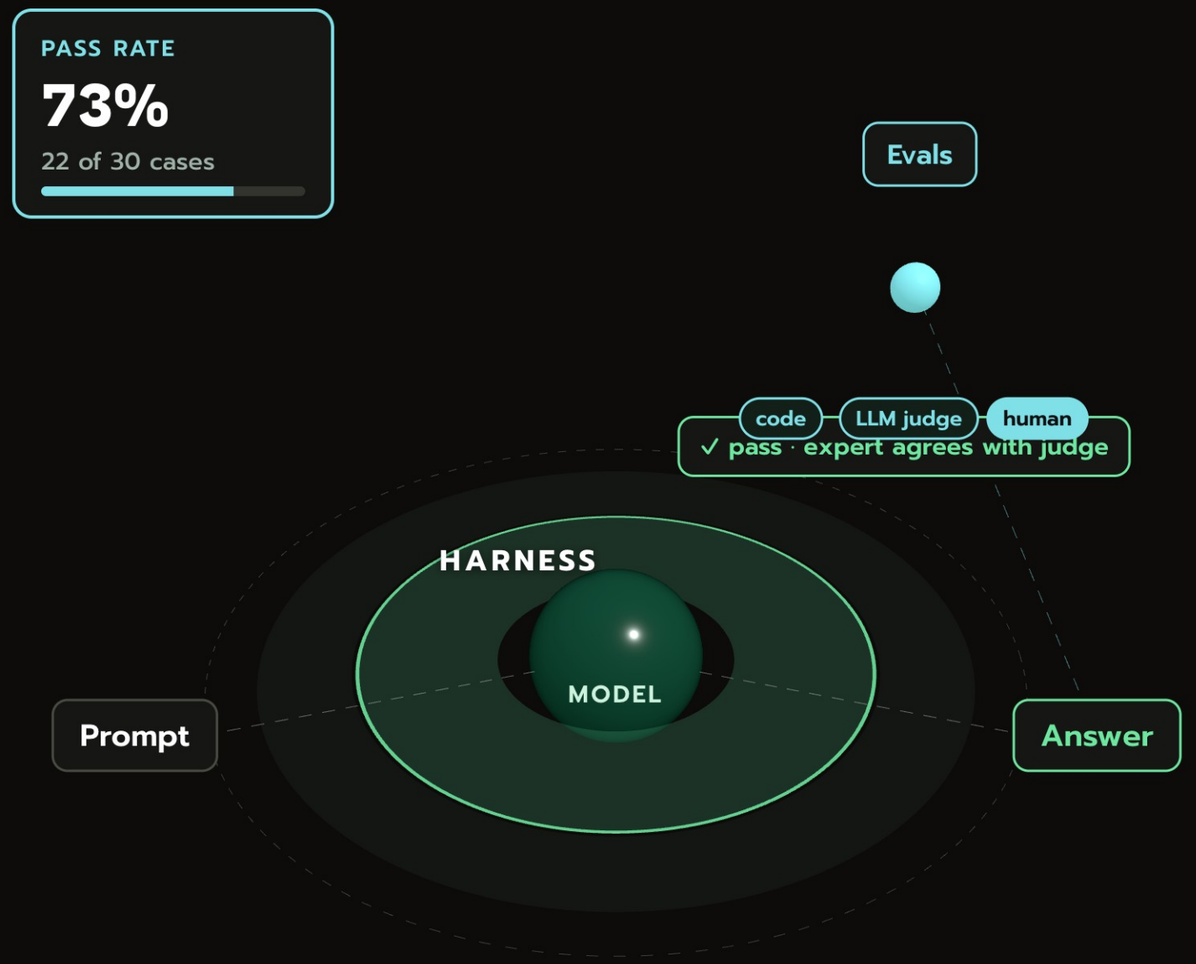
BEFORE RELEASE

Regression tests and experiments on fixed cases, against a baseline.

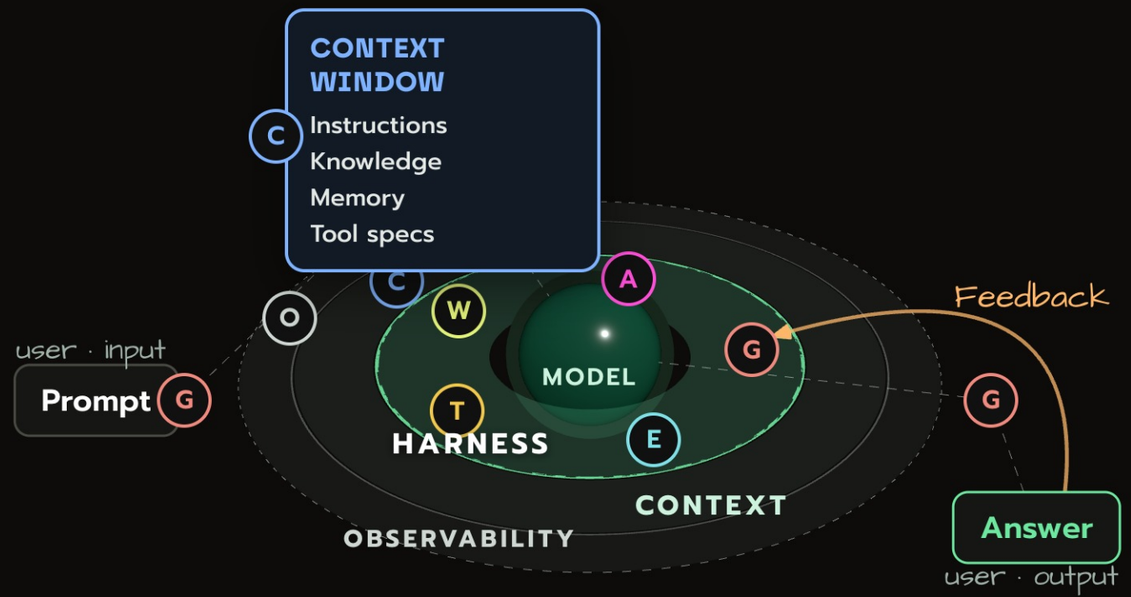
IN PRODUCTION

Samples of live traffic, judged and spot-checked by people.

Code first, a judge second, people to calibrate the judge.



Observability: what happened, how well, at what cost



TRACES what happened?

- guardrail
- context
- model
- tools · T W A G
- model
- eval
- guardrail
- answer

SPEED & ERRORS is it working?

latency	1.4 s
p95	2.0 s
errors	1

COST what does it cost?

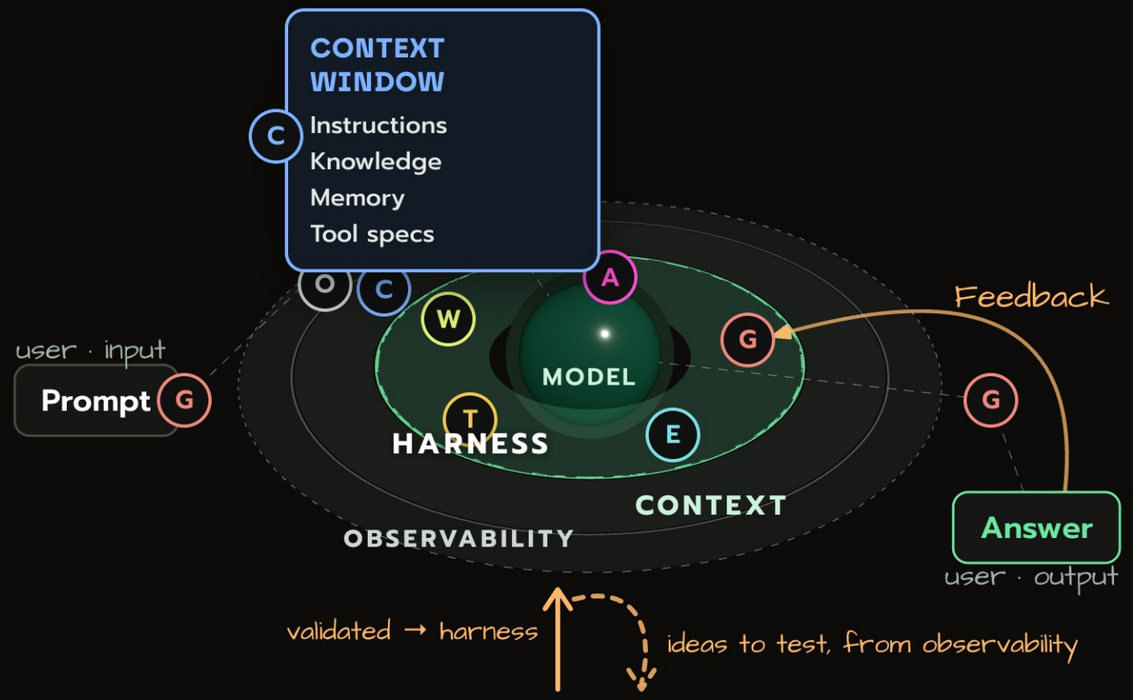
tokens	1,961
per request	€0.0082
total	€0.030

QUALITY is it right?

eval score	91%
👍	43
👎	7

Signals → dashboards → people decide what to fix → **ideas to test** → Experiments. Running is not the same as right: only **quality** shows it is right.

Experiments: compare before you ship



How do you systematically compare prompts, models, tools and agents before deployment?

Hypothesis · Dataset · Variants · Run · Evals · **Decide**

Kept fixed: cases, settings, judge and metrics. Keep a change only if it is better.

W **workflow v2** ↑ promoted — adds a check step — E **87%** vs 79% 👍

A **agent · new planner** ↑ promoted — fewer tool calls — E **91%** vs 85% 👍

T **new search tool** — hybrid retrieval — E **76%** vs 79% 👎

ONE IDEA TO KEEP

A GenAI system improves through loops, not launches.

Observability finds what to change; experiments prove it; the harness keeps it.

Improving the Performance of GenAI Systems

The four dimensions don't work alone: they connect through three feedback cycles.

1 Model Training and Adaptation

- Pre-training (and continued pre-training)
- Post-training (RLHF and other post-training techniques)

Parametric memory



C T W A G

2 Harness and Capability Extension

- Memory systems and artifacts
- Context and knowledge integration
- Capabilities and tools
- Orchestration and control
- Execution and evaluation

X E

3 Experiments and Validation

(offline)

- Benchmarks
- Datasets
- Test cases
- Metrics
- Evaluations

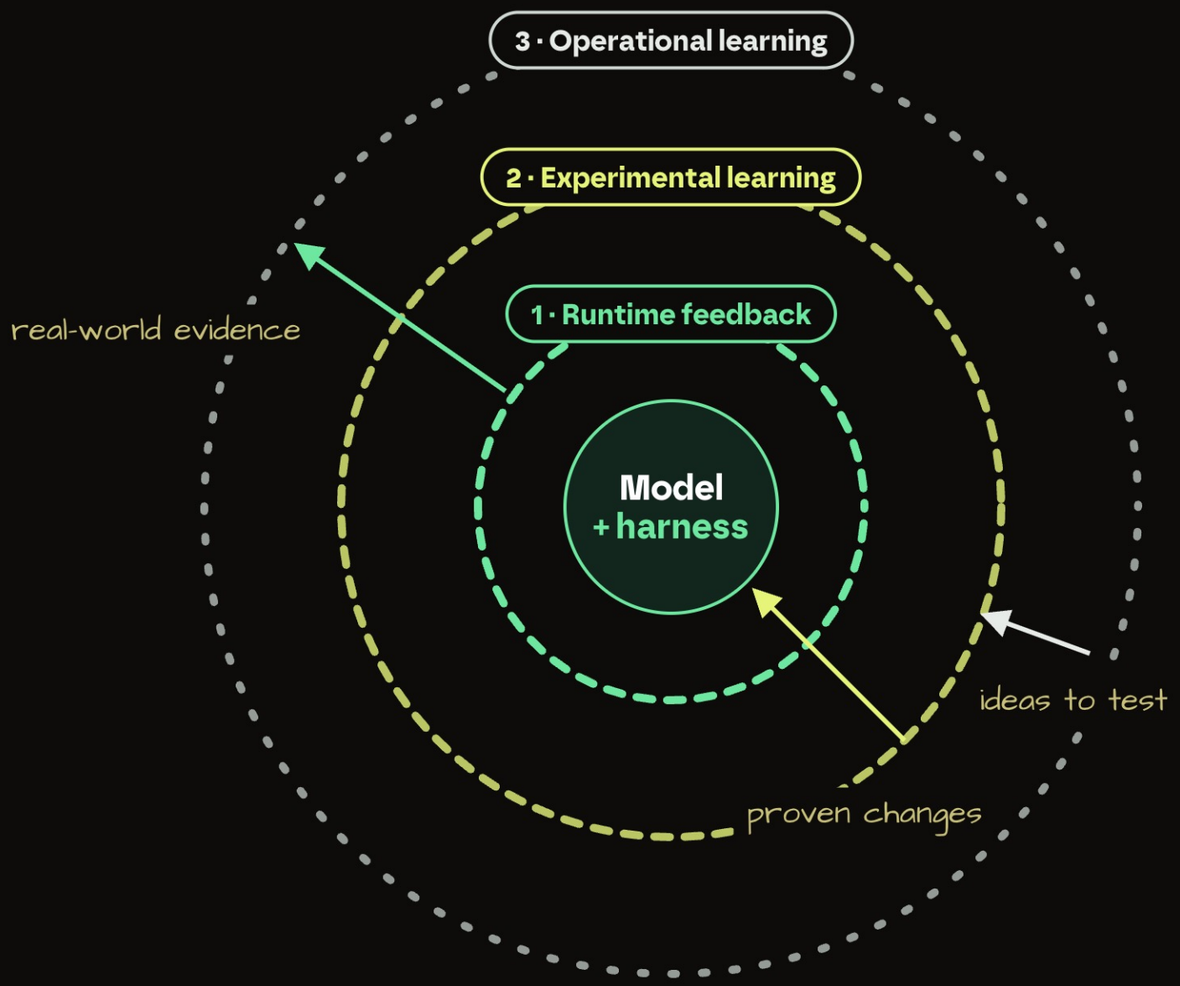
O E

4 Observability and Continuous Improvement

- Telemetry
- Traces
- Metrics and evaluators
- User feedback

CYCLES OF IMPROVEMENT

Three cycles, one system



G E

Runtime feedback

ONE TASK · SECONDS

Checks and corrects the work while it runs.

X E

Experimental learning

ONE CHANGE · DAYS

Tests a change and keeps it only if it is better.

◎

Operational learning

WHOLE SYSTEM · WEEKS

Finds patterns in real use and sets what to fix next.

Real use → **ideas to test** → experiments → **proven changes** → model or harness → new evidence → ...

REMEMBER

Observe in production, experiment before release: that loop is how GenAI systems improve.

Evidence in, validated change out, again and again.

WHERE YOU ARE

Six levels, L0 to L5

AI adoption isn't binary. Like self-driving cars, it moves in levels, and the higher you go, the more the improvement cycles must run on their own.

AUTOMATING WORKFLOWS

L0
AI as a Tool

Personal convenience; nothing reorganizes.

L1
Individual Automation

One person automates their own work.

L2
Team Automation

A team shares the workflow.

AGENTS IN YOUR ORG

L3
Directed Autonomy

You point an agent across the company.

L4
Self-Directed Autonomy

The agent notices and proposes.

L5
Self-Driving Enterprise

The agent acts; humans govern.

No fully self-driving organization exists yet.

Adapted from Ann Miura-Ko (Floodgate), six levels of enterprise AI adoption, modelled on the SAE levels of driving automation.

TO CLOSE

A GenAI system is never finished. It learns from experience: every interaction is evidence, and every validated change makes it better.

Models are trained once. Systems keep learning.